

Sessió 3

Desenvolupament Web Full Stack (React, Node, Mongo)

Curs 2026-2027

Pràctica: E-commerce. Sessió 3



1. Dates 3 2. Objectius 3 3. Feina prèvia abans de la sessió 3 4. Feina durant la sessió i abans de la següent sessió 4

4.1 Identificació d'entitats del projecte 4 4.2 Creació del diagrama de domini 4 4.3

Introducció als ADR (Architecture Decision Records) 4 4.4 Integració al repositori 5

Referències 5 Apèndix 6 A. Bones pràctiques en ADR 6 B. Errors comuns i depuració 6 C. Fitxer .env i bones pràctiques 6 D. Errors comuns i depuració 6 Lliurables de la Sessió 3 6

1. Dates

- Setmana 2: 29/09

2. Objectius

Pe s	Preferent?	Funcionalitat
0,4	✓	Modelatge del domini de l'aplicació (entitats i relacions)
0,3	✓	Introducció i redacció d'ADRs (Architecture Decision Records)
0,3		Documentació col·laborativa al repositori

3. Feina prèvia abans de la sessió

- Revisar conceptes de **bases de dades no relacionals** (NoSQL, MongoDB).
- Llegir què és un **ADR** i per a què serveix.
- Instalar una eina de diagrames (p. ex. [Draw.io](#), [Lucidchart](#) o VSCode amb extensió *Draw.io Integration*).

4. Feina durant la sessió i abans de la següent sessió

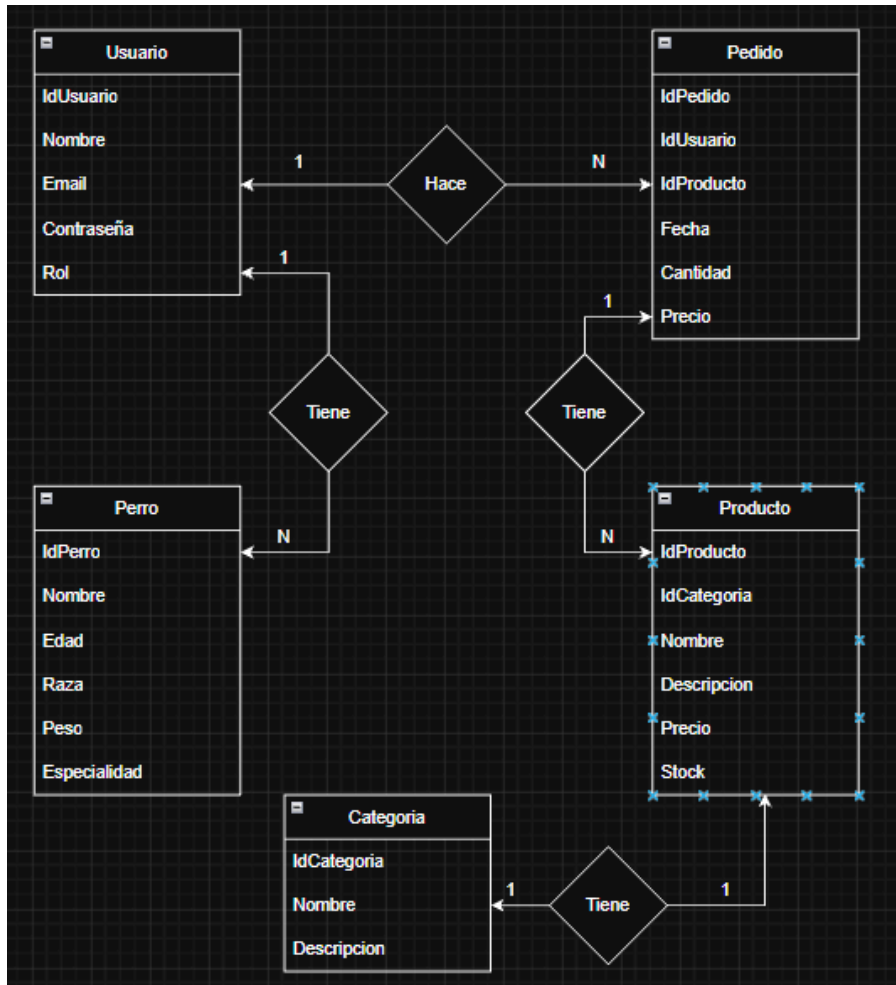
4.1 Identificació d'entitats del projecte

- Definir les entitats principals de l'e-commerce:
exemples:
 - **Usuari** (client, administrador)
 - **Producte**
 - ...
- Identificar **atributs bàsics** per cada entitat.
- Identificar **relacions** (p. ex. un usuari té moltes comandes, una comanda té molts productes).

4.2 Creació del diagrama de domini

- Crear un **diagrama conceptual** amb les entitats i relacions.
- Definir cardinalitats bàsiques (1..1, 1..N, N..M).
- Guardar el diagrama en format **.png** i afegir-lo al repositori backend a

/docs/diagrams/.



4.3 Introducció als ADR (Architecture Decision Records)

- Explicació del format ADR:
 - Títol
 - Context (per què prenem aquesta decisió)
 - Decisió (què fem exactament)
 - Conseqüències (positives i negatives)
- Exemple d'ADR inicial:
 - # ADR-001: Base de dades del projecte
 - ## Context
 - Necessitem una base de dades flexible per a emmagatzemar productes, usuaris i comandes amb relacions no rígides.
 - ## Decisió
 - Farem servir MongoDB com a base de dades principal, gestionada via Docker.
 - ## Conseqüències
 - + Flexibilitat per a nous camps i entitats.
 - + Bona integració amb Node/Express.
 - Menys adequat per a consultes molt relacionals.

```
1  # ADR-001: Elección de la base de datos
2
3  ## Contexto
4
5  El proyecto necesita una base de datos para almacenar la información de los usuarios, productos, categorías y pedidos.
6
7  ## Decisión
8
9  Utilizaremos MongoDB como base de datos principal del proyecto.
10
11  MongoDB se ejecutará mediante Docker para facilitar su instalación y configuración durante el desarrollo.
12
13  ## Consecuencias
14
15  ### Positivas
16
17  - Flexibilidad para modificar la estructura de los documentos.
18  - Buena integración con Node.js y Express.
19  - Fácil ejecución mediante Docker.
20  - Permite almacenar documentos con estructuras flexibles.
21
22  ### Negativas
23
24  - Las consultas con relaciones complejas pueden resultar menos adecuadas que en una base de datos relacional.
25  - Será necesario diseñar correctamente las relaciones entre los documentos.
```

```
# ADR-002: Estructura inicial del proyecto

## Contexto

El proyecto está compuesto por un frontend y un backend que forman parte de la misma aplicación.

Necesitamos decidir cómo organizar ambos componentes dentro del repositorio para facilitar el desarrollo y el mantenimiento del proyecto.

## Decisión

Utilizaremos un único repositorio con una estructura de monorepo.

El proyecto estará organizado de la siguiente manera:

- `frontend/`: aplicación frontend.
- `backend/`: aplicación backend.
- `docker/`: configuración relacionada con Docker.
- `.gitignore`: archivos y carpetas que no deben incluirse en Git.
- `README.md`: documentación general del proyecto.

## Consecuencias

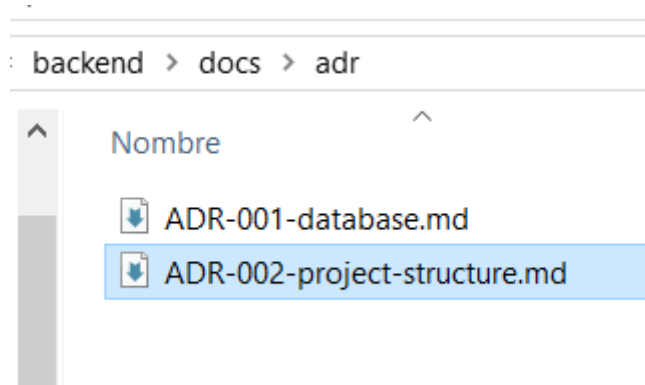
### Positivas

- Frontend y backend están centralizados en un mismo repositorio.
- Es más sencillo gestionar el proyecto completo.
- La estructura permite separar claramente frontend y backend.
- La configuración de Docker queda centralizada.

### Negativas

- El repositorio contiene tanto frontend como backend.
- El repositorio puede crecer más que si cada parte estuviera separada.
- Los cambios de frontend y backend se gestionan dentro del mismo repositorio.
```

- Crear carpeta `/docs/adrs/` al repositori.



- Redactar un mínim de 2 ADRs:
 - ADR-001: Elecció de base de dades.
 - ADR-002: Estructura inicial del projecte (monorepo o repos separats).

4.4 Integració al repositori

- Afegir diagrama i ADRs al repositori.
- Fer commit amb missatge:

```
git add docs/  
git commit -m "docs: add domain diagram and initial ADRs"  
git push origin main
```

Referències

- [What is an ADR? – Michael Nygard](#)
- [Awesome ADR – repositori amb exemples](#)
- [MongoDB – Documentació oficial](#)
- [Draw.io \(eina de diagrames gratuïta\)](#)

Apèndixs

A. Bones pràctiques en ADR

- Cada decisió té un **número incremental (ADR-001, ADR-002, ...)**.
- Guardar-los en text pla (**.md**) al repositori.
- Escriure de forma clara i breu, no més d'1 pàgina.

B. Errors comuns i depuració

- Diagrames massa tècnics → en aquesta fase només conceptuals. •
- ADRs buits o poc concrets → cada ADR ha de justificar una decisió real. •
- No actualitzar ADRs quan canviïn les decisions → mantenir-los vius.

C. Fitxer **.env** i bones pràctiques

- Centralitzar secrets (usuari, password Mongo).
- Afegir **.env** al **.gitignore**.

D. Errors comuns i depuració

- Ports ocupats.
- Volumes sense permisos.
- Variables d'entorn mal definides.

Lliurable de la Sessió 3

Diagrama de domini i mínim 2 ADRs al repositori del backend ([/docs/](#)).